

贷后信贷风控与处置

信衡 CreditSentry

从风险预警到授信处置的

多 Agent 可举证自主闭环

银行不缺预警系统。缺的是预警之后那一段——谁去取证、凭什么下结论、**敢不敢自动执行、出了事怎么向监管交代。**

这份材料里的每一个数字都来自可复现的运行，每一条约束都由回归测试强制。跑一遍需要 5 条命令、零第三方依赖、零云账号、零网络。

这份材料怎么读

按评审四项要求组织。每一部分的结论都落到具体文件与具体验证方式，不使用无法检验的表述。

01 场景与价值 目标场景从哪来、给谁用、痛点是什么、输入输出是什么、能不能复制到别的行业。 第 3—6 页	02 方案设计 五层架构、Agent 分工的推导过程、协作流程、上下文传递、结果验证、异常分支怎么兜。 第 7—13 页	03 Skill 与集成 16 个 Skill 的分层与规范、可迁移的复用内核、MCP / RAG / 可观测的集成路径。 第 14—17 页	04 可行性与落地 评估指标与实测数据、已可运行的 Demo、安全边界、风险控制、开源计划与边界坦白。 第 18—21 页		
1+1+6 Manager · Team Leader · Worker	16 Skill（领域自研 12）	4 MCP Server	12 路由规则 v1.4.0	993 回归 + 评估用例全绿	14,262 行代码 · 零第三方依赖

一条贯穿全文的判断：**不可复现就不可审计**。因此流程骨架用确定性状态机，模型只在阶段内做判断——这决定了后面所有设计。

01

场景与价值

目标场景来源、目标用户、核心痛点、真实需求、业务价值、输入输出与行业可复制性。

场景不是选出来的，是从一条断裂的作业链上找出来的

商业银行小微与对公信贷的贷后管理环节。预警系统银行早就有了，问题出在预警之后。

① 预警产生 行内预警平台 · 外部舆情 · 监管推送	② 核实取证 人工逐条跨系统查	③ 风险定性 人工判断	④ 出处置方案 人工撰写	⑤ 审批 风险经理 / 审批人	⑥ 执行与留痕 人工操作 + 事后补材料
--------------------------------	--------------------	----------------	-----------------	--------------------	-------------------------

深色三段 = 本方案接管的范围。① 银行已有系统，⑤ 必须留给人——这两处刻意不碰。

目标用户是谁 主用户 · 贷后风险经理：每天面对成百上千条预警，逐条核实，处置周期以天计。 协同方 · 客户经理：只有他能向客户索取财务报表这类不在任何系统里的材料。 监督方 · 合规与内审：他们不看结论，看的是「这个结论怎么来的」。	为什么是这个场景 它同时具备三个特征，缺一不可：高频（日均千级预警）、高摩擦（每条要跨 4 个以上系统取数）、高后果（处置动作直接动客户额度）。 前两个决定了自动化有价值；第三个决定了不能直接放手让模型干——这正是 Agent Infra 该解决的问题形态。	我们不做什么 不做评分卡与违约概率预测——那是 GBDT 的活，做了也是重复造轮子。 不做「AI 预警」。银行的预警数量已经过剩，再加一个模型只会让噪声更多。 我们做的是预警之后那段没人管的链路。
---	---	--

四个痛点，四条对应设计——每条都写明了证伪条件

痛点来自贷后作业的真实结构。右列不只给机制，还给「如果它不成立，会怎么被看出来」。

痛点	现状	真实需求	机制与证伪条件
预警过载	日均千级预警，重复推送与例行提醒占大头，客户经理逐条人工核实	要的不是更准的预警，是把噪声压下去且不漏高危	信号归并去重 + 降噪， 每条被丢弃的信号都记录丢弃理由 。 证伪：若被丢弃信号无法回溯，「不漏高危」就是空话
取证割裂	征信、司法、工商、流水分属四套系统，口径不一、授权各异	统一取数并把证据固定下来，事后能翻回原件	MCP 统一接入 + 证据账本（只可追加、sha256、强/弱/缺失分级）。 证伪：若原件调不出或哈希对不上，证据链不成立
敢不敢自动执行	压降额度、追加担保直接影响客户，出错即投诉与问责，于是宁可全人工	要分级：哪些能自主、哪些必须人批、哪些系统永远不碰	四维执行闸门 L0—L3，纯规则、失败即阻断。 证伪：865 组全组合遍历中只要有一次不可逆动作落进 L0/L1，设计即失败
无法举证	结论写在报告里，依据散在聊天记录与个人经验中，内审时无法重建当时的判断	每个结论都要能指回具体证据与具体规则	结论强制挂载证据编号；路由决策本身即一条可读轨迹（含规则 ID 与版本）。 证伪：出现无源结论时报告应当拒绝生成，而不是打印出来

这四条里，**第三条是真正的门槛**。前两条是效率问题，第三条是责任问题—— 责任问题解决不了，前两条做得再好，系统也只能停在「给人看的建议」。

输入输出：一条预警进去，一套可举证的处置材料出来

下面是系统的对外契约。左边是它需要什么，右边是它必须交付什么——两边都是硬约定。

输入

预警信号

来源、主体、信号类型、时间、详情（可多条、可重复）

主体标识

统一社会信用代码 / 行内主体号

取数授权

征信查询授权编号。缺失则该项直接判为证据缺口，**不降级绕过**

决策时点

可选。回溯场景下冻结时点，只喂入该日之前的信息

输出 · 每次运行必产出 9 个文件

处置意见书

正文每处结论带证据角标，可点开看原件

审计报告

合规项逐条举证 + 风险模式沉淀

证据账本

只可追加，含哈希、来源、证据等级与定级理由

全链路轨迹

含路由决策与裁决依据两类专有记录

取数留痕

含被拒调用——越权尝试也必须留痕

运行指标

降噪率、证据充分度、质疑覆盖率、用量与成本

结局 A · 风险成立

出分级处置方案。需审批的**停下来等人**，批准后执行并回执，最后由独立角色审计核验。

结局 B · 风险不成立

出「维持原状 + 加强监测」并留痕。「**我们看过并决定不动**」必须有据可查，不能表现为什么都没发生。

结局 C · 未能定论

出**取证任务清单**转人工：为什么停、已查到什么、还缺什么、每项找谁补。**不给风险结论**——证据不足时给结论就是编。

业务价值分两层：省下来的时间，和敢不敢用

第一层是效率，容易量化但不稀缺；第二层是可举证性，难量化但决定这套系统能不能真的上生产。

第一层 · 效率（实测，可复现）

- 单案端到端 **18 毫秒**（规则推理） / 接大模型时以秒计。对照：人工处置周期以天计
- 降噪率 **50 — 62.5%**（三条实测链路），且每条丢弃都可回溯
- 单案成本 **¥0.0153**；按日均 2000 件推算约 ¥31/天、¥7,628/年
- 自动取数 单案 **30+ 次**跨 4 系统调用，全部自动完成并留痕

成本口径取自真实的上下文装配结果而非经验估值；不含修复轮——该发生率要真机跑过才有实数。

第二层 · 可举证性（决定能不能用）

- 每个结论都能指回证据**——无源结论会让报告拒绝生成，而不是打印出来。
- 每次分支都能说清为什么**——路由决策带规则 ID 与版本，不靠日志拼凑。
- 越权尝试也留痕**——只在成功路径留痕的系统，没法回答「有没有人试过绕过去」。
- 不可逆动作系统永不执行**——这一条让「自动化」在信贷场景变得可谈。

行业可复制性：可迁移的是内核，不是流程

保险核赔	供应链金融	工业设备运维	内容与账号风控
报案归并去重 → 单证取证 → 定损定性 ⇔ 反欺诈 质疑 → 分级赔付（小额自动、大额人核）	贸易背景真实性核验：多源单据交叉验证 + 对抗质疑，处置动作换成放款闸门	告警降噪 → 多源取证 → 根因定位 ⇔ 反驳 → 分级处置（远程复位可自主、停机必须人批）	信号聚合 → 证据固定 → 判定 ⇔ 申诉视角质疑 → 分级处置（限流可逆、封禁不可逆）

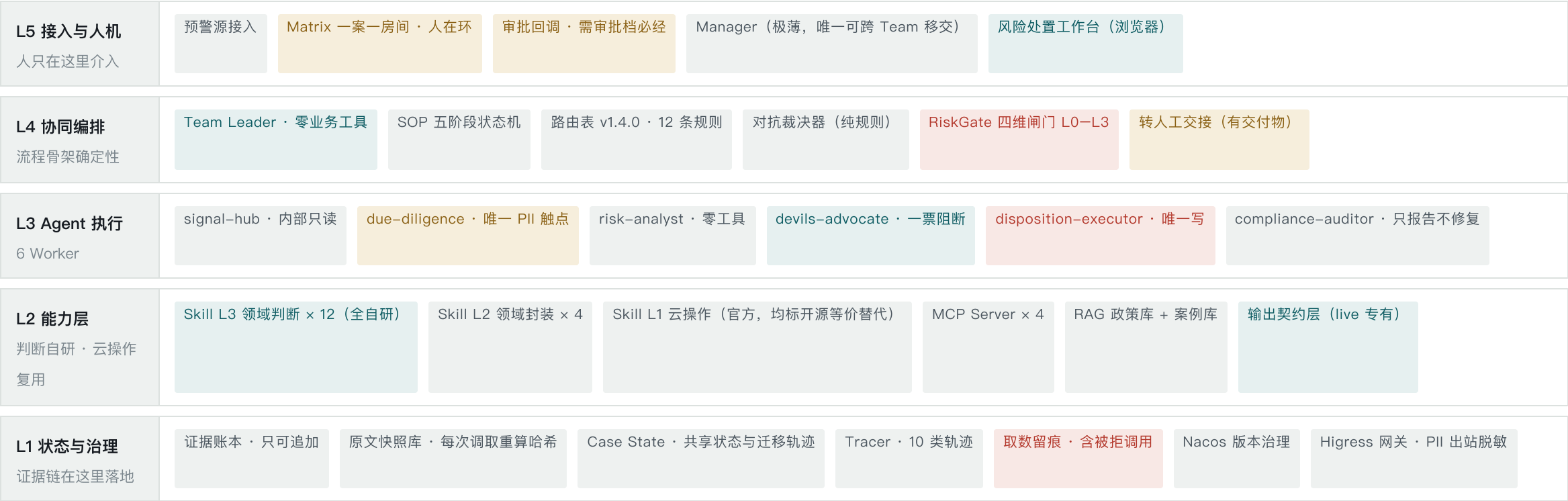
四者共享同一组骨架：信号归并 → 证据固定 → 对抗式判定 → 按可逆性分级处置 → 留痕审计。换行业要改的是取数接口与判定口径；架构、闸门、账本、审计不动。可迁移清单见第 16 页。

方案设计

系统架构、Agent 分工、任务拆解、协作流程、上下文传递、结果验证与异常分支处理。

五层架构：人在哪里介入、谁在编排、谁在干活、留下什么痕迹

橙色 = 人工必经点，红色 = 有副作用或不可逆的位置。全图只有这几处——这是刻意收敛的结果，不是画漏了。



贯穿三层的脊柱：一份权限矩阵

同一份声明同时驱动四处——运行时的取数授权、Skill 授权、Worker 配置与身份文件的生成、以及回归断言。测试逐字节比对磁盘上的配置与由矩阵生成的结果，因此「权限矩阵即部署配置」是可校验的事实，不是修辞。

Agent 数量不是拍出来的，是数出来的

判据两条，且可证伪：权限集不可安全合并则必须拆；目标函数相互对立则必须拆。反过来——权限相同 + 目标一致 + 失败策略一致，则必须合并。

等价类 01

内部只读聚合

signal-hub

只读行内数据，不触外部、不下结论

等价类 02

唯一外部 PII 取证触点

due-diligence

触点唯一，脱敏范围才可收敛可验证

等价类 03 · 唯一拆分处

零工具纯推理

risk-analyst
⇔ 目标函数对立
devils-advocate

两者权限集完全相同。拆分理由不是权限差异，而是同一上下文内模型会自治坍缩——自己质疑自己不成立

等价类 04

唯一写触点

disposition-executor

全系统唯一能改额度的角色，且不做自由推理

等价类 05

唯一轨迹读 + 知识库写

compliance-auditor

看得见全链路，但动不了任何业务系统

合并 02 + 03 会怎样

定性者持有取证工具 → 边查边下结论的确认偏差；且 PII 触点扩散，脱敏范围失控

合并 定性 + 质疑 会怎样

同一上下文自我质疑 → 模型倾向自治，对抗坍缩，质疑形同虚设

合并 04 + 05 会怎样

执行者自审 → 直接违反《商业银行内部控制指引》不相容职务分离

为什么不是 10 个？把取证官再拆成征信官 / 司法官 / 流水官，三者权限集相同、目标一致、失败策略一致——按合并判据必须合并。继续拆只增加消息跳数与 Token，不增加任何控制力。这条法则可证伪：只要能举出一对「权限与目标都不同却应当合并」的角色，法则就被推翻。

协作流程：模型只做阶段内判定，不决定流程走向

金融场景要求可复现可审计，而自由编排不可复现——同一个案子重跑十次可能走出七条路径。所以流程骨架是确定性状态机，12 条规则显式声明。

INTAKE 受理 signal-hub 归并降噪 · 敞口测绘	EVIDENCE 取证 due-diligence 跨 4 系统取原文并登记证据	ADJUDICATION 裁决 定性 ⇄ 质疑 强制并行，质疑不可跳过	DISPOSITION 处置 闸门定级；需审批档停下等人	AUDIT 审计 核验生效 · 合规逐条举证 · 经验回流	CLOSED 闭环 归档
-------------------------------------	---	--	---------------------------------	----------------------------------	-----------------

三条硬约束写在路由表里，不交给模型

R-04 质疑不可跳过：取证充分后强制并行派发定性与质疑，没有「跳过」这条分支。

R-05 唯一回退边：全流程只有一条回退边，且带重试上限，不存在无限循环。

R-03 / R-12 两条转人工入口：材料不够走 R-03；证据够了但裁决反复收敛不了走 R-12。

为什么要有 R-12

R-03 只管住了「证据充分度不达标」。但裁决判证据不足时，充分度可能是达标的——质疑器失效、清单覆盖不全都会这样。此时回退会再次命中 R-04，形成环路。

规则推理的确定性掩盖了这个环，接真实模型后它就会出现。这是接大模型过程中发现并补上的第 12 条规则。

可审计性的落点

路由决策本身就是一条轨迹记录，携带三个字段：路由键、命中规则 ID、规则版本。

「为什么当时走了这条分支」不是靠日志拼凑推测出来的，是直接读出来的。

12 条规则在代码与声明式配置中同构声明，测试断言两者不漂移，且每条规则都必须可被触达——死规则会让测试失败。

这一层的取舍很明确：牺牲编排的灵活性，换取整条链路的可复现与可举证。在信贷处置场景，这个交换是划算的——灵活性带来的收益远小于「说不清为什么」带来的风险。

上下文传递：三个坑，逐个用代码堵

上下文工程最容易出问题的不是「装不下」，而是装丢了却看不出来。三条机制都留痕，不留痕的堵法等于没堵。

坑一 · 静默截断

超预算时悄悄丢尾部，模型看不到关键事实却照样给结论。

堵法：裁剪必须留痕——丢了什么、为什么丢，都记进装配账单并进轨迹。必需块不参与裁剪；必需块本身超预算则直接抛错，不硬塞。

坑二 · 原文灌入

裁判文书全文进上下文，既爆预算，又让「模型看的是哪一份」无法举证。

堵法：证据只放编号 + 抽取字段，检测到长文本直接抛错。原文靠哈希锚定，给人看不给模型看——两者本就该分开。

坑三 · 关键块被挤掉

预算压力下把清单、输出契约裁掉，模型不知道自己要做完什么。

堵法：排序原则是目标与契约在前、事实在中、参考在后。模型对长上下文首尾更敏感，把「做完的标准」挤到中段是最糟的情况。

待办清单：不是提示技巧，是完成性契约

清单之所以有效，是把长程任务外化成可枚举、可检查的子项。但它有个根本弱点：清单通常由模型自己维护，模型可以偷偷改、悄悄划掉。

所以本方案的清单是：由系统从上游产物派生 → 注入提示词告知完成标准 → 模型返回后由代码逐项核对。提示词里的清单只是告知契约，真正起作用的是事后那次核对。

这堵住了一个真实漏洞：此前质疑方可以只反驳 3 条主因里的 1 条、对另外 2 条保持沉默，系统无法区分「反驳失败」与「根本没看」。

未被质疑的主因 ≠ 质疑通过。前者只是没人看过。

六维查询改写与澄清选路

立场（求证方找构成要件，证伪方找认定门槛）· 术语规范化（行内口语→监管术语）· 信号主题 · 条款直查（精确引用不走语义）· 否定式（仅证伪方，主动找除外与豁免）· 时效。

时效维修复了一个真实缺陷：2017 年的回溯案件曾召回 2025 年才生效的行内制度。

这是知识维度的前视污染，比证据层更隐蔽——条款看起来「一直都在」。

澄清不默认追问用户：能自动决定的自动决、能派任务的派任务、确需人判断的给选项不问开放问题。理由很实际——银行场景补充信息的成本不在用户打字，在于去哪个系统查、有没有授权。

结果验证：两道独立的关，一道管「对不对」，一道管「敢不敢做」

前者是对抗式判定，后者是纯规则闸门。两者刻意分开——判断错了不该直接变成动作。

第一道 · 对抗式判定

定性岗与质疑岗输入完全相同、上下文完全独立、目标函数刻意对立：一个论证风险成立，一个负责推翻它。

异构约束：两侧必须绑定不同技术路线的模型。拆成两个角色是为了避免自洽坍缩——若共用同一批权重，坍缩只是从上下文层面搬到权重层面。配成同源会被拒绝启动。

裁决用纯规则，不用模型：裁决是责任分配的时刻，必须可复现、可复核。规则是「证据等级优先于置信度」。

质疑覆盖率参与裁决：覆盖不满 100% 即判为质疑未完成，走向退补证——这是失败即阻断，不是失败即放行。

第二道 · 四维执行闸门

风险等级 × 证据等级 × 敞口金额 × 可逆性 → 四档：

L0 只读诊断，全自主

L1 轻度干预，全自主

L2 影响授信，必须人工审批后执行

L3 不可逆或大额，只出方案，系统永不执行

三条铁律：纯规则引擎（安全判定不依赖网络与模型发挥）、失败即阻断而非放行（任何入参缺失或规则未命中一律降为 L3）、不可逆动作恒为 L3（无论风险多高、证据多强、金额多小）。

同一个动作，不同情境下的档位不同——这才叫真的在分级

额度压降 30% · 敞口 620 万

→ L2 审批后执行

同一动作 · 敞口 2 亿

→ L3 只出方案，系统不执行

审批被驳回

→ 降级 L0 额度未被修改，可在取数留痕里核对

865 组全组合遍历证明：不可逆动作在任何入参组合下都不会落进 L0/L1。这不是抽样测试，是穷举——闸门是安全核心，抽样过不了关。

异常分支：三条失败路径，降级方向恒为阻断

一个系统的可信度不看它顺利时的表现，看它出问题时往哪个方向倒。下面三条都是实测可复现的路径。

失败一 · 模型给不出合规输出

定性失败 → 判「证据不足」回退补证
质疑失败 → 判「质疑未完成」直接阻断

只有不影响决策的环节才允许降级。质疑器坏了不等于质疑通过——与闸门的失败即阻断是同一条原则在不同层的体现。

降级必须留痕：进轨迹、进日志、进指标，并在界面显式打出来。一次悄悄降级的运行和一次正常运行长得一样，那么「这个结论怎么来的」就永远说不清。

失败二 · 取证不足以定论

回退补证 ≤ 2 轮 → 仍不足 → 转人工，且必须有交付物

转人工不等于甩给人。系统放弃自动处置时的最后一个动作，是把工作交接清楚：为什么停、已查到什么、还缺什么、每项找谁补。

待补项区分「系统能重取的」与「只能靠人要的」——财务报表不在任何可查系统内，系统重试多少次都拿不到，只能由客户经理向客户索取。这正是转人工存在的理由。

失败三 · 人工审批被驳回

L2 → 降级 L0 只读诊断，额度不被修改

审批是真的阻塞：后端线程停在那里等决策，不是走个过场的确认框。

驳回后可在取数留痕里核对——没有任何一次成功的写操作。这条由回归断言，不靠人工检查。

接大模型后新增的一层：输出契约

规则推理模式下模型输出由代码构造，字段必然齐全；接真实模型后它由模型生成，没有任何东西保证它齐全。因此 live 路径多一层固定顺序的处理链：

传输重试 限流与 5xx 退避重试；鉴权失败立即上抛	抽 JSON 剥围栏与寒暄，只处理包装不猜内容	归一化 吸收表述差异	结构校验 枚举、区间、必填、证据引用非空	语义校验 证据编号是否真实存在、清单是否逐项回执	至多一轮修复 回喂错误让模型自己改	失败策略 见上方三条
-------------------------------	----------------------------	---------------	-------------------------	-----------------------------	----------------------	---------------

归一化与校验之间有条硬线：把 "62%" 改成 0.62 是纠正表述；给一条没有证据的结论编一个 ID 是伪造证据。前者做，后者绝不做——这条线由专门的反向断言守着。

Skill 与工具集成

核心 Skill 设计、复用价值、工具与企业系统的集成路径，以及 MCP、RAG、可观测的能力规划。

01 场景与价值 第 3—7 页	02 方案设计 第 8—14 页	03 Skill 与集成 第 15—18 页	04 可行性与落地 第 19—21 页
---------------------	---------------------	---------------------------	------------------------

Skill 分三层：领域判断力自研，通用云操作复用

分层依据是一句话——「这件事换个行业还成立吗」。不成立的必须自研，成立的没必要重复造。

L3 领域判断 · 12 个，全自研

SignalFusion · ExposureMapping · CreditReportProbe · LitigationProbe · TxnFlowAnalyze · GuaranteeProbe · QueryRewrite · RiskRootCause · RiskGate · ComplianceCheck · SafeDisposition · PostmortemDistill

承载信贷风控领域知识与监管条款绑定。**这些不可能复用现成能力**——「单笔涉诉标的占敞口 ≥5% 且未结案才构成偿债能力信号」这种口径，只能自己写并绑定条款版本。

L2 领域封装 · 4 个，自研外壳

EvidenceLedger · PolicyRag · CaseMemory · ReportCompose

语义自研，底层存取复用官方能力。例如证据账本的分级规则、只可追加约束、哈希校验是自研的；对象存储与索引则直接用云能力。

L1 云操作 · 直接用官方

对象存储 · 向量检索 · 日志 · 监控

不重复造轮子。**全部标注开源等价替代**（MinIO / pgvector / Loki / Prometheus），PoC 默认走替代——这样评委在本地就能跑，不需要云账号。

CreditSkill Spec：让 Skill 可分发、可评估、可回滚

每个 Skill 交付三件套：说明文档 + 输入输出 Schema + 评估集。三者都由代码里的元数据生成，因此文档不会与实现漂移。

每个 Skill 显式声明：触发条件、依赖、失败策略、安全等级、绑定的监管条款、评估集规模、复用价值。

评估集是发布门禁——不全绿不允许灰度发布。

覆盖度差额如实记录

906 组用例全绿，但其中 865 组来自闸门的全组合遍历。**还有 9 个 Skill 的评估集仍是种子集、未接执行器。**

这个差额写在清单里，未接执行器的如实记为 SKIPPED，而不是把「没跑」粉饰成「通过」。

把差额写在明处，比事后被追问更经得起看。复赛计划把评估集扩到 100+ 标注案例。

复用价值：五个与业务解耦的内核，可以单独拿走

判断一个组件有没有复用价值，标准是「把信贷两个字去掉，它还成立吗」。下面五个成立，因此计划单独开源。

内核	它解决的通用问题	换到别的场景怎么用	开放形式
Agent 拆分法则	多 Agent 系统「该拆几个」没有判据，全靠拍脑袋	任何多 Agent 系统都适用：按权限等价类推导数量，目标函数对立时才额外拆	方法论文档 · CC BY 4.0
EvidenceLedger 证据账本	「结论有据可查」缺少可执行的落地形式	只可追加 + 哈希锚定 + 证据分级 + 无证据即拒绝断言。 最通用的一个 ，任何需要举证的系统都能用	组件 · Apache-2.0
RiskGate 四维执行闸门	Agent 敢不敢执行有副作用的动作	把「风险等级 × 证据等级 × 影响面 × 可逆性」换成本行业的四个维度即可；失败即阻断的骨架不动	组件 · Apache-2.0
CreditSkill Spec	Skill 没有统一规范，无法分发、评估与回滚	三件套 + 失败策略 + 评估门禁的规范本身与领域无关	规范 · Apache-2.0

还有一项计划开源的数据集

风险案例回放数据集 (CC BY 4.0)：含**误报陷阱样例**与反事实配对。

这个领域目前没有现成 benchmark——公开数据集全是「tabular 特征 → 违约标签」，服务的是评分卡。而我们要的标签是「**在那个时点、依据那些证据，该不该处置**」，不是「这家企业后来违约了没有」。两者性质不同。

复用的边界在哪

不可复用的是**判定口径**——「涉诉标的占敞口 5%」这类阈值绑定具体监管条款与行内制度，换行业必须重写。

可复用的是**骨架与约束**：怎么拆 Agent、怎么固定证据、怎么分级执行、怎么留痕。

把这条边界说清楚，比声称「全面可复用」更有用。

集成路径：Mock 与真实接入零 Schema 差异

Demo 用 Mock 数据是常态，风险在于「Demo 能跑、接真系统要重写」。我们用结构约束消除这个风险。

MCP · 4 个 Server，17 个工具

- 信贷核心 额度、敞口穿透、押品、担保台账（读）+ 调额、追加担保、回滚（写）
- 人行征信 征信报告、期间变动比对、被查询记录 **需授权编号，缺失即拒绝，不降级绕过**
- 司法工商 涉诉检索、裁判文书原文、工商登记、变更历史
- 账务流水 流水明细（自动分片）、对手方汇总、异常模式识别

替换点只有一处：把进程内直连换成经网关的 MCP 客户端。工具名称、入参 Schema、返回结构、错误码完全不变——Agent、Skill、路由表与契约测试均无需改动。

调用统一收敛在一处，因此三件事只实现一次：权限校验、审计留痕、失败策略。 **只有限流类错误值得重试**，权限与授权类错误立即上抛，不重试不降级。

RAG · 两库四能力

- 知识源 监管政策库（条款级）+ 历史处置案例库
- 查询改写 六维子查询，各带独立过滤与权重；每条命中记录由哪几维召回
- 时点过滤 **条款生效日 ≤ 案件时点**，防知识维前视污染
- 结果融合 多维召回后排序融合，空集时区分「库里没有」与「被时效过滤掉了」
- 经验回流 确认的风险模式写回案例库，**可追溯哪次案件产生了哪条经验**

沉淀的不只是「什么算风险」，还有「什么不算」——风险模式里专门有一条反例字段。

可观测：10 类轨迹记录

对齐 OpenTelemetry GenAI 语义约定，另加**两类本方案特有的记录**：**路由决策**（含路由键、规则 ID、规则版本）与**对抗裁决**（含双方结论、分歧点、采信理由）。

属性命名保持与 OTel 一致，接真实 Collector 时只需替换记录器，不改调用方。

治理与网关规划

Nacos：Skill、提示词、路由表版本化，支持标签灰度与一键回滚。

Higress：统一出口、凭证透传、PII 出站脱敏。Worker 只持消费方令牌，**真实密钥留在网关侧，Worker 拿不到也换不掉**。

配置里凭证一律是引用而非明文，因此配置可进版本库、密钥不进。

这一层的实现进度

已实现：4 个 Mock Server（含限流、授权缺失等错误码）、统一授权与留痕、10 类轨迹落盘、凭证引用四种形态。

未实现：真实行内系统接入；Nacos 与 Higress 尚在配置层预留，网关侧未落地。

可行性与落地计划

评估指标、可运行 Demo、安全边界、风险控制、开放与开源计划，以及明确的边界。

01 场景与价值 第 3—7 页	02 方案设计 第 8—14 页	03 Skill 与集成 第 15—18 页	04 可行性与落地 第 19—21 页
---------------------	---------------------	---------------------------	------------------------

它现在就能跑，而且跑的东西可以被逐条复核

五条命令，零第三方依赖、零云账号、零网络，数秒内完成且全绿。下面所有数字都来自这几条命令的真实输出。

74 安全边界回归全绿	906 Skill 评估用例全绿	13 接大模型的故障模式全绿	3 端到端链路覆盖三种结局	36 可逐条复核的验收断言	0 第三方运行时依赖
----------------	---------------------	-------------------	------------------	------------------	---------------

三条链路，覆盖三种不同的闸门结局

案件	场景	结论与档位
链路 A	三类信号相互印证，逐条质疑均不成立	风险成立 · 需审批档 压降 30% + 追加担保
链路 B	信号类型完全相同，但个案层面均不具实质性	风险不成立 · 只读档 维持原状 + 加强监测
链路 C	真实历史回溯：2017 年担保圈风险传染，时点冻结在 2017-04-01	风险成立 · 只出方案档 系统不执行

链路 B 才是技术难点：A 与 B 是同一类主体、完全相同的三类信号，差别只在个案细节。让系统敢说「这不是风险」，比让它报警难得多——误杀正常客户会直接抽贷断贷、引发投诉与监管问责。

链路 C 是时点冻结回溯：只喂入 2017-04-01 之前的公开信息，系统判定风险成立（净未覆盖代偿敞口达直接敞口 2.02 倍）；历史结局是 31 个月**后**债券违约——该结局在数据准备时就被摘走，全程不可达，仅用于事后评分。

已可运行的 Demo：风险处置工作台

```
python3 poc/serve.py --open
```

浏览器里选一笔预警点「开始处置」，七个可现场操作的动作：

- ① 点开任一证据**翻开原件**并当场重算校验码
- ② 并排看定性 ⇄ 质疑的逐条分歧
- ③ 审批时**后端线程真的停在那里**等你点按钮
- ④ 驳回 → 降级只读，额度未被修改
- ⑤ 改一个决定性案情要素 → **当场翻案**
- ⑥ 敞口改 2 亿 → 同一动作从需审批升到只出方案
- ⑦ 让零权限岗位去改额度 → **被拒且这次尝试也留痕**

工作台与命令行是同一套代码、同一份产出，没有为演示另造一条链路。新增的只有三个旁路钩子，且它们抛错一律吞掉——可视化坏了不能反过来弄坏业务链路，这条由回归断言。

安全边界与风险控制：约束由代码强制，不由文档承诺

下面每一条都对应一段可执行的断言。其中带「含反向验证」的，不只测「合法输入能通过」，还测「非法输入真的会被拒」——只测正向的约束等于没有约束。

权限与职责边界

- 写权限唯一：只有一个角色能改额度
- PII 触点唯一：只有一个角色能读征信与流水
- 定性岗零工具权限（防边查边下结论）
- 执行方与审计方不是同一角色
- 编排者不持有任何业务工具
- 越权调用一律拒绝，且被拒调用进留痕
- 部署配置与权限矩阵零漂移，漂移即测试失败

执行与数据边界

- 不可逆动作在任何入参组合下都不落进自主档
- 入参缺失时降为最高管控档，而非放行
- 白名单外动作一律拒绝
- 幂等重放不重复执行；回滚可恢复原值
- 审批被拒即降级只读，额度不被修改
- 无证据 / 引用无效证据 / 仅凭缺失证据的结论一律被拒
- 原文不入模型上下文；裁剪必须留痕
- 配置文件不含任何明文密钥（按密钥形态匹配）

模型使用边界

- 对抗双方必须异构，配成同源即拒绝启动（含反向验证）
- PII 触点的模型不得越出数据驻留白名单（含反向验证）
- 无模型入口的角色不得绑定任何模型档位
- 两种推理模式产出同一套输出契约
- 归一化只吸收表述差异，绝不伪造证据引用（含反向验证）
- 模型失效时降级方向恒为阻断
- 密钥只存在本次运行的内存里，不入配置、不入任何留痕

回溯伦理边界

只回溯已有公开定论的历史事件（已违约 / 已重整 / 已结案），不对当前存续且未定论的主体输出风险结论。新闻报道只用于定位案件与时间线，不进证据账本——这恰好符合我们自己的证据等级规则：新闻是典型的弱证据。

无前视信息由三道防线保证：每条证据标注首次公开日并逐条校验；历史结局在数据构造时即被摘出，任何环节结构上不可达；知识召回按时点过滤。

数据来源披露

合成数据 + 公开数据，不含任何真实银行客户数据。链路 A/B 为全合成；链路 C 逐字段标注来源：担保关系与金额取自公开披露，单家银行敞口为推导值（从不公开），征信与流水依法不可公开故为合成。

不是「编了一个数据集」，而是用真实公开事件做骨架，只在法律禁止公开的两个模态上做受约束的合成。

边界：把没做的写在明处

这一页是这份材料里最该被仔细读的一页。把差额写清楚，比事后被追问更经得起看。

已完成，可现场复核

6 Worker 的身份与配置声明 · SOP 五阶段状态机与 12 条路由 · 16 个 Skill 的 Schema 与实现 · 4 个 Mock MCP Server · 证据账本与原文快照 · 模型配置四层分离与四条不变量 · 三条端到端链路（含一条真实历史回溯）· 10 类轨迹落盘 · 提示词版本治理 · 上下文装配器 · 质疑与取证双清单 · 六维查询改写 · **接大模型的输出契约层与失败策略执行点** · 转人工交接单 · **风险处置工作台** · 74 项安全回归 + 906 组评估 + 13 个故障模式

尚未做，如实列出

- 真实行内系统接入（4 个 Server 仍为 Mock，但接口契约已按零差异设计）
- 生产级并发与多租户隔离
- 9 个 Skill 的评估集仍为种子集，覆盖度差额已在清单中记录
- Nacos 与 Higress 仅在配置层预留，网关侧未落地
- 容器与 K8s 部署给出了替换点，未实机验证
- **接大模型已在故障注入伪端点上跑通全部 13 个模式与完整链路，但尚未在真实商业端点上跑过——那需要 Key，而 Key 在使用者手里**

一处已知的标定局限

敞口升档阈值目前对所有客户分层使用**同一套绝对值**。在链路 C 这类大型集团客户上， 它会把几乎所有可逆动作都推到最高档。

机制是对的，标定不对——复赛应把阈值改为按客户分层参数化。

这是跑真实案例才暴露出来的问题，如实记在这里而不是悄悄调参掩盖。

一个贯穿全文的取舍

我们在每一处都选择了**牺牲灵活性换取可举证性**：流程用确定性状态机而非自由编排； 裁决用纯规则而非模型； 闸门在入参缺失时降级而非放行； 模型失效时阻断而非尽力而为。代价是这套系统不够“聪明”——它不会自己想出新流程，也不会 在证据不足时给你一个看起来合理的结论。但在信贷处置这个场景，这个交换是划算的。

最后一句：这套系统的价值不在于它跑得多快，而在于它出问题时往哪个方向倒。 我们把所有失败路径都调成了「停下来、说清楚、交给 人」，而不是「尽力而为地跑完」。

开放、开源与复赛计划

开源的挑选标准是「把信贷两个字去掉，它还成立吗」——成立的才有被别人拿走的价值。

开放与开源计划		
组件	说明	协议
Agent 拆分法则	与业务解耦的方法论：Agent 数量由权限等价类推导	CC BY 4.0
EvidenceLedger	证据账本，六个组件里最通用的一个	Apache-2.0
RiskGate	四维执行闸门内核，失败即阻断的骨架	Apache-2.0
CreditSkill Spec	Skill 规范三件套 + 失败策略 + 评估门禁	Apache-2.0
4 个 MCP 接口契约	金融取数接口的工具定义与错误码，含契约测试用例	Apache-2.0
风险案例回放数据集	含误报陷阱样例与反事实配对	CC BY 4.0
这个领域目前没有现成 benchmark——公开数据集全是「特征 → 违约标签」，服务的是评分卡；而我们要的标签是「在那个时点、依据那些证据，该不该处置」。		

复赛计划

- 接入真实 Nacos，做 Skill 与路由表的灰度回滚演练
- Higress 网关落地凭证透传与 PII 出站脱敏
- 评估集扩至 100+ 标注案例，建立反事实配对集
- 回滚与越权拦截的故障注入演练——现有 13 个模型侧故障模式扩展到系统侧
- 敞口阈值改为按客户分层参数化
- Manager 变厚——扩展至「贷后 + 反欺诈 + 法务」三 Team 协同

主指标：配对翻转率

同一主体、只改一个决定性因子，结论必须随之翻转。

这个指标比准确率难得多，也诚实得多——它无法靠「一律报警」或「一律放行」刷分，两种偷懒策略的翻转率都是 0。

如果系统在一对样本上给出相同结论，说明它压根没看那个因子——不管整体准确率多高，这都是失败。

```
现在就可以复核  poc/serve.py --open 工作台七个动作 | poc/run_demo.py --all 三条链路 | poc/test_safety.py 74 项安全回归 | tools/run_evals.py 906 组评估 |
tools/check_pit.py 时点冻结 | tools/live_conformance.py 13 个故障模式（无需 Key、无需联网）
```